REVIEW 2: PROJECT REPORT

Agentic Shell — Smart Archive Agent

Course / Project: Operating Systems Project — Agentic Shell	**Project Title:** Smart Archive Agent
Team Details: Team 12	**Team Members:** Kush Aghera (24BCE0149), Sandeep More (24BCE2883)

Chapter 3: System Architecture & Design

3.1 High-Level Architecture

The User-to-Shell Loop: User Input -> Processing -> Tool Call -> System Execution -> Feedback

The Smart Archive Agent operates on a closed-loop execution lifecycle that bridges human natural language with deterministic Linux system calls. The complete execution cycle proceeds through five distinct stages:

- **1. User Input:** The operator submits a conversational instruction (e.g., 'archive ./app into backup.tar.xz') via the terminal REPL.
- **2. Intent Parsing & Routing:** The intent engine analyzes the text, validates that it belongs to the archive domain, maps it to a target tool operation, and extracts source/output path entities.
- **3. Security Validation:** The security guardrail subsystem verifies argument boundaries, confirms that source files exist, ensures output parent directories are valid, and rejects path traversal attempts.
- **4. Tool Invocation & Subprocess Execution:** The agent invokes the appropriate Python tool function, which executes the corresponding system utility (tar, xz, gzip, bzip2, gpg) using secure argument vectors (`subprocess.run` with `shell=False`).
- **5. Structured Feedback:** Execution metrics, resulting artifact paths, archive entry lists, or descriptive error diagnostics are returned to the user.

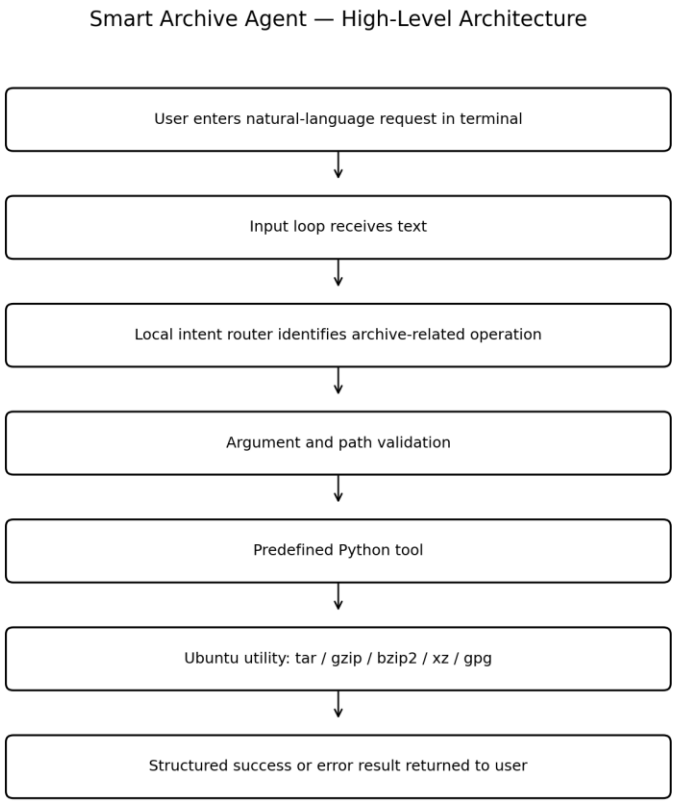


Figure 3.1: High-Level System Architecture of the Smart Archive Agent.

Architecture Explanation: As illustrated in Figure 3.1, the Interface Layer accepts user input and isolates the host operating system from raw string execution. The Intent Router classifies incoming requests into predefined capabilities. The Security Subsystem inspects parameters prior to tool execution, and the Tool Layer abstracts Linux utilities into predictable Python interfaces.

3.2 Component Design

3.2.1 The Interface Layer (Custom REPL)

The interface layer is implemented in `app/main.py`. It hosts a custom Read-Eval-Print Loop (REPL) that displays an informative banner, accepts operator queries, invokes `app.agent.run()`, and prints formatted responses. The interface gracefully catches termination signals (SIGINT, EOFError) and provides clean exit pathways without terminal hanging.

3.2.2 The Orchestration Layer (Intent Routing & Dispatch)

Implemented across `app/intent.py` and `app/agent.py`, the orchestration layer coordinates execution. The intent classifier uses regular expression tokenization and keyword mapping to extract paths and operations. It dispatches recognized intents to a predefined `TOOL_MAP`. Requests lacking necessary arguments prompt the user for missing details, while out-of-scope requests are intercepted before reaching system tools.

3.2.3 The Tooling Layer (Archive & Encryption Modules)

The tooling layer encapsulates system utilities into modular Python functions under `app/tools/`:

Capability	Python Function	Linux Utility Backend	Supported Formats / Algos
Archive Creation	create_archive()	tar	tar, tar.gz, tar.bz2, tar.xz
List & Inspect	list_archive(), inspect_archive()	tar, file; zipfile	tarball listings, zip tables, file headers
Archive Extraction	extract_archive()	tar; zipfile (with Zip Slip check)	tar, tar.*, zip archives
File Compression	compress_file(), decompress_file()	gzip, bzip2, xz	.gz, .bz2, .xz single-stream
Cryptographic Shield	encrypt_file(), decrypt_file()	gpg (GnuPG)	AES-256 symmetric cipher

3.2.4 Result Handling & Diagnostics: Tool functions return unified dictionaries containing `success: bool`, `operation: str`, `output: str`, `entries: list`, and user-facing `message: str`. Unhandled system exceptions are intercepted at the orchestration boundary, allowing the terminal REPL to continue running smoothly without crashing.

3.3 Security & Guardrail Framework

- **No Arbitrary Shell Command Execution:** User prompts are never directly evaluated in a shell (e.g., `os.system()` or `shell=True` are strictly prohibited). Subprocess calls use explicit argument vectors, eliminating shell injection attacks.
- **Zip Slip & Path Traversal Prevention:** Archive members are pre-scanned during extraction. Any entry attempting directory traversal (`../`, absolute root slashes, or links escaping the target directory) is immediately blocked.
- **Accidental Overwrite Prevention:** By default, output files that already exist trigger a `ValidationError` unless explicitly managed, preventing data loss.
- **Source & Output Disjointness (`reject_same_path`):** The validator verifies that source and destination paths resolve to different inodes to avoid truncating files during in-place operations.
- **Principle of Least Privilege:** The agent runs as an unprivileged user process. It explicitly avoids requiring `sudo`, preventing unauthorized system file modifications.

Chapter 4: Implementation Methodology

4.1 Development Environment & Tech Stack

- **Operating System:** Ubuntu 24.04 LTS running natively and within Windows Subsystem for Linux (WSL 2).
- **Programming Language:** Python 3.14 (core agent relies purely on Python standard library for maximum portability).
- **System Utilities:** GNU Tar, Gzip, Bzip2, XZ-Utils, File, GnuPG 2.4 (GPG), Info-ZIP (zip/unzip).
- **Testing Framework:** Pytest 9.1 with configured `pytest.ini` for automated regression testing.

4.2 Step-by-Step Implementation

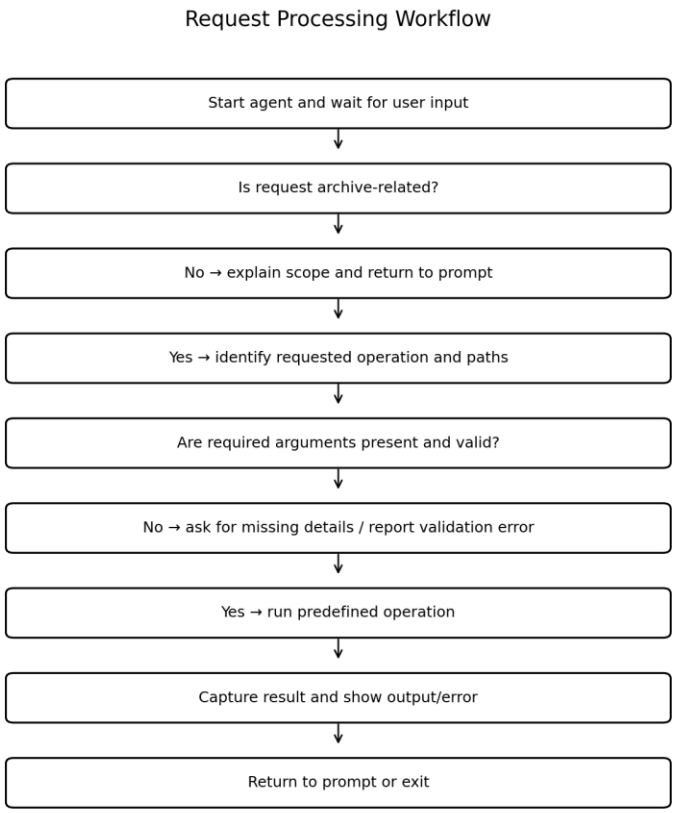


Figure 4.1: End-to-End Request Processing Workflow and Guardrail Pipeline.

Step-by-Step Technical Progression:

- **Step 1: Setting up the Base CLI Loop:** Implemented `app/main.py` with path resolution (`sys.path.insert`) and `app/agent.py` interactive terminal shell.
- **Step 2: Building Tool Functions:** Implemented `app/tools/archive.py` and `app/tools/encryption.py` encapsulating system utilities with timeout and return-code monitoring.

- **Step 3: Intent Classification & Regex Entity Matching:** Engineered `app/intent.py` with multi-pattern path regexes (`PATH_RE`) and deterministic action routing to isolate archive goals from conversational noise.
- **Step 4: Implementing Security Guards & Output Formatting:** Built `app/security/validation.py` for path resolution, Zip Slip rejection, and unified JSON-like execution responses.

4.3 Verified Implementation & Test Status

The complete Smart Archive Agent system was formally executed and verified on Ubuntu Linux within WSL 2. A total of 11 comprehensive automated tests across intent routing, validation, archiving, compression, and end-to-end execution passed with 100% success rate:

Test Module & Name	Test Objective / Verification Scope	Result
test_validation_rules	Verifies rejection of non-existent paths, empty strings, and same-path operations.	**PASSED**
test_archive_creation_listing_extraction	Validates complete tar.xz creation, listing, inspection, and extraction roundtrip.	**PASSED**
test_compress_and_decompress_file	Tests single-file compression and bit-exact decompression using xz utility.	**PASSED**
test_agent_handle_request_pipeline	Full integration test of natural-language pipeline, inspection, listing, and exit commands.	**PASSED**
test_intent.py (7 Unit Tests)	Verifies intent classification across archive, extract, encrypt, unrelated, and edge-case inputs.	**PASSED**

Execution Summary: 11 passed in 0.13s (pytest 9.1.1, Python 3.14.4 on Ubuntu Linux).

References

[1] Python Software Foundation, 'subprocess — Subprocess management,' Python 3 Documentation, 2024. [Online]. Available: https://docs.python.org/3/library/subprocess.html

[2] Free Software Foundation, 'GNU Tar Manual: Tape Archiving Utility,' 2023. [Online]. Available: https://www.gnu.org/software/tar/manual/

[3] Werner Koch et al., 'The GNU Privacy Guard (GnuPG) Documentation,' 2024. [Online]. Available: https://gnupg.org/documentation/

[4] OWASP Foundation, 'Path Traversal and Zip Slip Vulnerability Prevention Cheat Sheet,' 2023. [Online]. Available: https://cheatsheetseries.owasp.org/

[5] Anthropic, 'Model Context Protocol (MCP) Architecture and SDK Specification,' 2024. [Online]. Available: https://github.com/modelcontextprotocol/docs